

從 App Engine Users 服務遷移至 Cloud Identity Platform (模組 21)

來源：<https://codelabs.developers.google.com/codelabs/cloud-gae-python-migrate-21-idenplat?hl=zh-tw>

步驟數：8

瞭解如何遷移 Python 2 App Engine NDB & 將使用者服務應用程式遷移至 Cloud NDB (&A)；再升級至 Python 3

目錄

1. [1. 總覽](#)
2. [2. 背景](#)
3. [3. 設定/準備工作](#)
4. [4. 更新設定](#)
5. [5. 修改應用程式程式碼](#)
6. [6. *Python 2 回溯移植](#)
7. [7. 摘要/清除](#)
8. [8. 其他資源](#)

1. 總覽

[無伺服器遷移工作站系列程式碼研究室](#) (自學式實作教學課程) 和[相關影片](#)，旨在協助 [Google Cloud 無伺服器](#)開發人員完成一或多項遷移作業 (主要是從舊版服務遷移)，進而翻新應用程式。這樣做可提高應用程式的可攜性，並提供更多選項和彈性，讓您整合及存取更多 Cloud 產品，並更輕鬆地升級至新版語言。雖然一開始的重點是早期 Cloud 使用者，主要是 [App Engine](#) (標準環境) 開發人員，但本系列涵蓋範圍廣泛，也包括其他無伺服器平台，例如 [Cloud Functions](#) 和 [Cloud Run](#)，或適用於其他平台。

本程式碼研究室旨在向 Python 2 App Engine 開發人員說明如何從 [App Engine Users API/服務](#)遷移至 [Cloud Identity Platform](#) (GCIP)。此外，從 App Engine NDB 遷移至 [Cloud NDB](#) 以存取 Datastore (主要涵蓋在遷移模組 2 中) 時，也會**隱含遷移**，並升級至 Python 3。

第 20 個單元說明如何將 Users API 的使用方式新增至第 1 個單元的範例應用程式。在本單元中，您將採用完成的第 20 個單元應用程式，並將其使用方式遷移至 Cloud Identity Platform。

未使用 Users 服務嗎？

如果您的應用程式未使用 App Engine Users 服務，請略過第 20 到 21 個單元，或將這些程式碼研究室當做練習，熟悉如何將 Users 遷移至 Cloud Identity Platform。

在接下來的研究室中

- 將 [App Engine Users 服務](#)替換為 [Cloud Identity Platform](#)
- 將 [App Engine NDB](#) 的使用方式替換為 [Cloud NDB](#) (另請參閱第 2 節)
- 使用 Firebase Auth 設定不同的驗證身分識別提供者
- 使用 Cloud Resource Manager API 取得專案 IAM 資訊
- 使用 Firebase Admin SDK 取得使用者資訊
- 將範例應用程式移植到 Python 3

軟硬體需求

- 擁有 [Google Cloud Platform 專案](#)，且已啟用 [GCP 帳單帳戶](#)
- Python 基礎技能
- 熟悉常見的 Linux 指令
- 具備[開發](#)及[部署](#) App Engine 應用程式的基本知識
- 可正常運作的 Module 20 App Engine 範例應用程式

問卷調查

2. 背景

App Engine Users 服務是使用者驗證系統，供 App Engine 應用程式使用。這項服務提供 Google 登入做為身分識別提供者，並提供方便的登入和登出連結供應用程式使用，同時支援管理員使用者和僅限管理員使用的功能。為提升應用程式可攜性，Google Cloud 建議從舊版 App Engine 套裝組合服務遷移至 Cloud 獨立服務，例如從 Users 服務遷移至 Cloud Identity Platform 等。

Identity Platform 以 [Firebase Authentication](#) 為基礎，並新增多項企業功能，包括多重驗證、OIDC 和 SAML 單一登入支援、多用戶群架構、99.95% 服務水準協議等。如要瞭解這些差異，請參閱 [Identity Platform](#) 和 [Firebase 驗證產品比較頁面](#)。這兩項產品的功能都遠多於使用者服務。

本第 21 堂課的程式碼研究室會示範如何將應用程式的使用者驗證從 Users 服務切換至 Identity Platform 功能，盡可能模擬第 20 堂課中示範的功能。第 21 堂課也介紹如何從 App Engine NDB 遷移至 Cloud NDB，以存取 Datastore，重複第 2 堂課的遷移作業。

雖然「模組 20」的程式碼「宣傳」為 Python 2 範例應用程式，但來源本身與 Python 2 和 3 相容，即使在模組 21 中遷移至 Identity Platform (和 Cloud NDB) 後，仍維持這種相容性。升級至 Python 3 時，您可以繼續使用 Users 服務，因為遷移至 Identity Platform 是選用功能。請參閱第 17 個單元的程式碼研究室和影片，瞭解如何升級至 Python 3 等第 2 代執行階段，同時繼續使用隨附的服務。

將應用程式遷移至 Python 3，然後視需要回溯至 Python 2

開發人員將無伺服器應用程式現代化時，主要目標是升級至 Python 3。由於相容性問題，本教學課程會從 Users 遷移至 Identity Platform，並升級至 Python 3。遷移至 Identity Platform 和 Python 3 後，我們會回顧已完成的第 21 個模組應用程式，並將其回溯移植到 Python 2，其中包含額外步驟。如果使用者必須繼續使用 Python 2，我們才建議將程式碼向後移植到 Python 2。

本教學課程包含下列步驟：

1. 設定/準備工作
 2. 更新設定
 3. 修改應用程式程式碼
-

3. 設定/準備工作

本節將說明如何：

1. 設定 Cloud 專案
2. 取得基準範例應用程式
3. (重新) 部署及驗證基準應用程式
4. 啟用新的 Google Cloud 服務/API

這些步驟可確保您從可運作的程式碼開始，並準備好遷移至獨立的 Cloud 服務。

1. 設定專案

如果您已完成第 20 個單元的程式碼研究室，請重複使用該專案 (和程式碼)。或者，您也可以[建立全新專案](#)，或重複使用其他現有專案。確認專案已啟用帳單帳戶和 App Engine 應用程式。找出專案 ID，並在進行本程式碼研究室時隨時備妥，每當遇到 `PROJ_ID` 變數時，請使用該 ID。

2. 取得基準範例應用程式

其中一項必要條件是可正常運作的第 20 課 App Engine 應用程式，因此請完成該程式碼研究室 (建議做法；連結如上)，或從存放區複製[第 20 課程式碼](#)。無論使用你的或我們的，我們都會從這裡開始 (「START」)。本程式碼研究室會逐步說明遷移作業，最後提供的程式碼會與第 21 堂課存放區資料夾 (「FINISH」) 中的程式碼類似。

- 開始：[Module 20 資料夾](#) (Python 2)
- 完成：模組 21 資料夾 ([Python 2](#) 或 [Python 3](#))
- [整個存放區](#) (複製或下載 [ZIP 檔案](#))

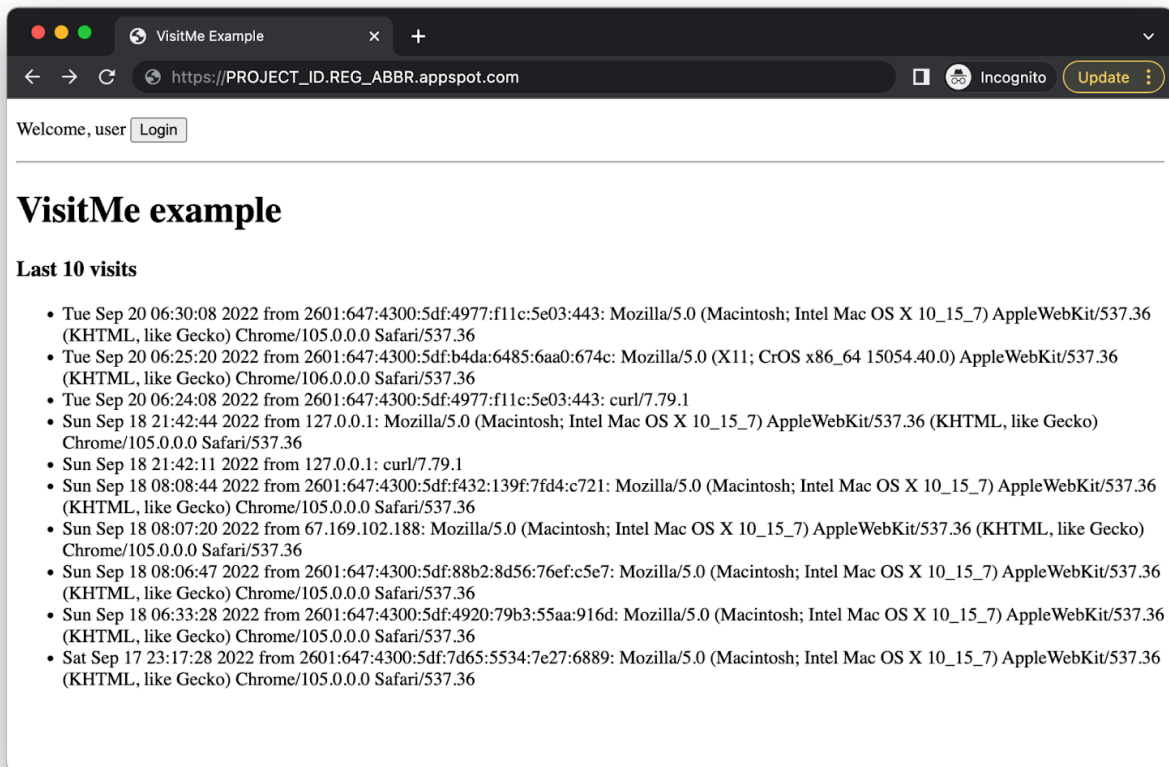
複製「Module 20」存放區資料夾。輸出內容應如下所示，如果您已完成第 20 模組的程式碼研究室，可能會有 `lib` 資料夾：

```
$ ls
README.md          appengine_config.py  templates
app.yaml           main.py              requirements.txt
```

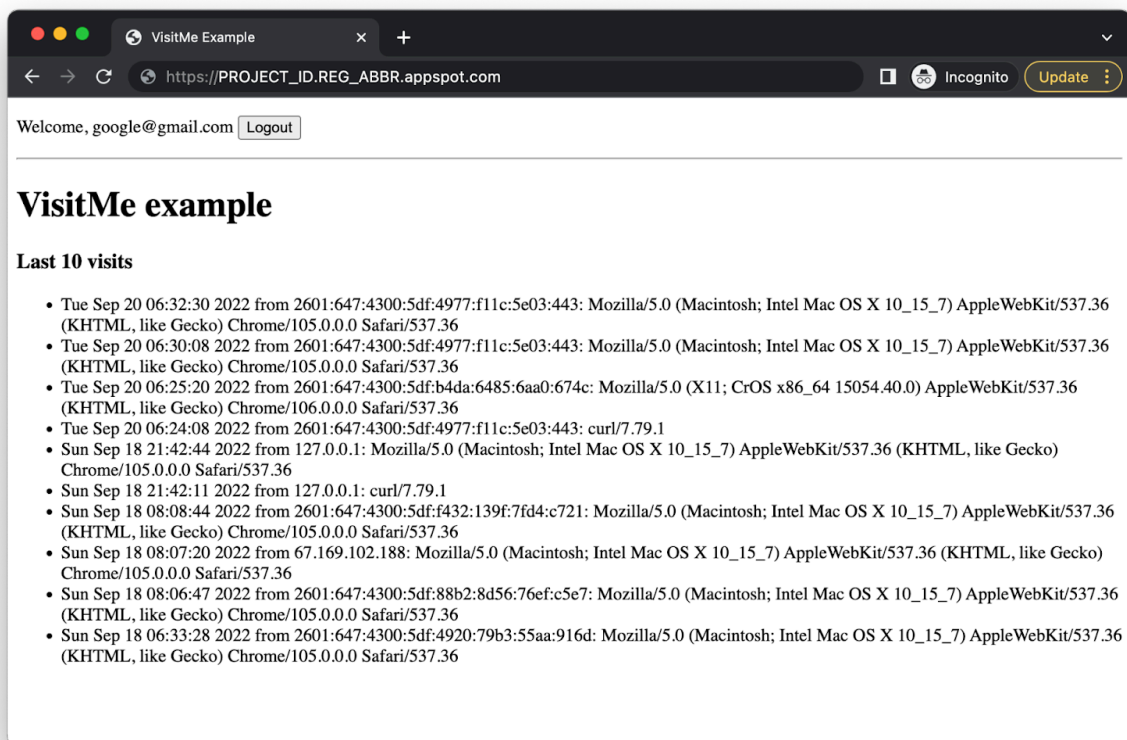
3. (重新) 部署及驗證基準應用程式

請按照下列步驟部署 Module 20 應用程式：

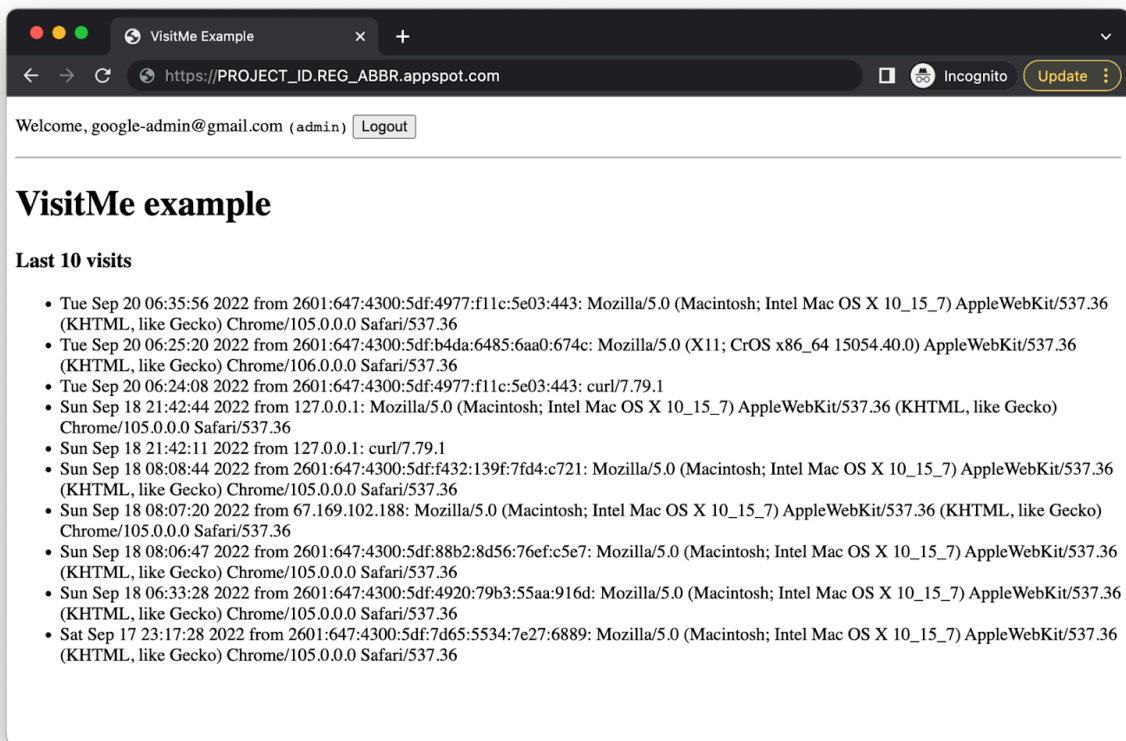
1. 刪除 `lib` 資料夾 (如有)，然後執行 `pip install -t lib -r requirements.txt` 重新填入內容。如果同時安裝 Python 2 和 3，可能需要使用 `pip2`。
2. 確認您已[安裝並初始化](#) `gcloud` 指令列工具，並已瞭解[其用法](#)。
3. 如果不想在每次發出 `gcloud` 指令時輸入 `PROJ_ID`，請先使用 `gcloud config set project PROJ_ID` 設定 Cloud 專案。
4. 使用 `gcloud app deploy` 部署範例應用程式
5. 確認應用程式是否正常執行，且未發生錯誤。如果您已完成第 20 個單元的程式碼研究室，應用程式會在頂端顯示使用者登入資訊 (使用者電子郵件地址、可能的「管理員徽章」和登入/登出按鈕)，以及最近的造訪記錄 (如下圖所示)。



以一般使用者身分登入時，系統會顯示使用者的電子郵件地址，「登入」按鈕也會變更為「登出」按鈕：



以管理員使用者身分登入時，系統會顯示使用者的電子郵件地址，旁邊會加上「(管理員)」：



4. 啟用新的 Google Cloud API/服務

簡介

第 20 堂課的應用程式使用 App Engine NDB 和 Users API，這些是隨附的服務，不需要額外設定。但獨立的 Cloud 服務則需要設定，更新後的應用程式會同時採用 Cloud Identity Platform 和 [Cloud Datastore](#) (透過 [Cloud NDB 用戶端程式庫](#))。此外，我們需要判斷 App Engine 管理員使用者，因此也必須使用 [Cloud Resource Manager API](#)。

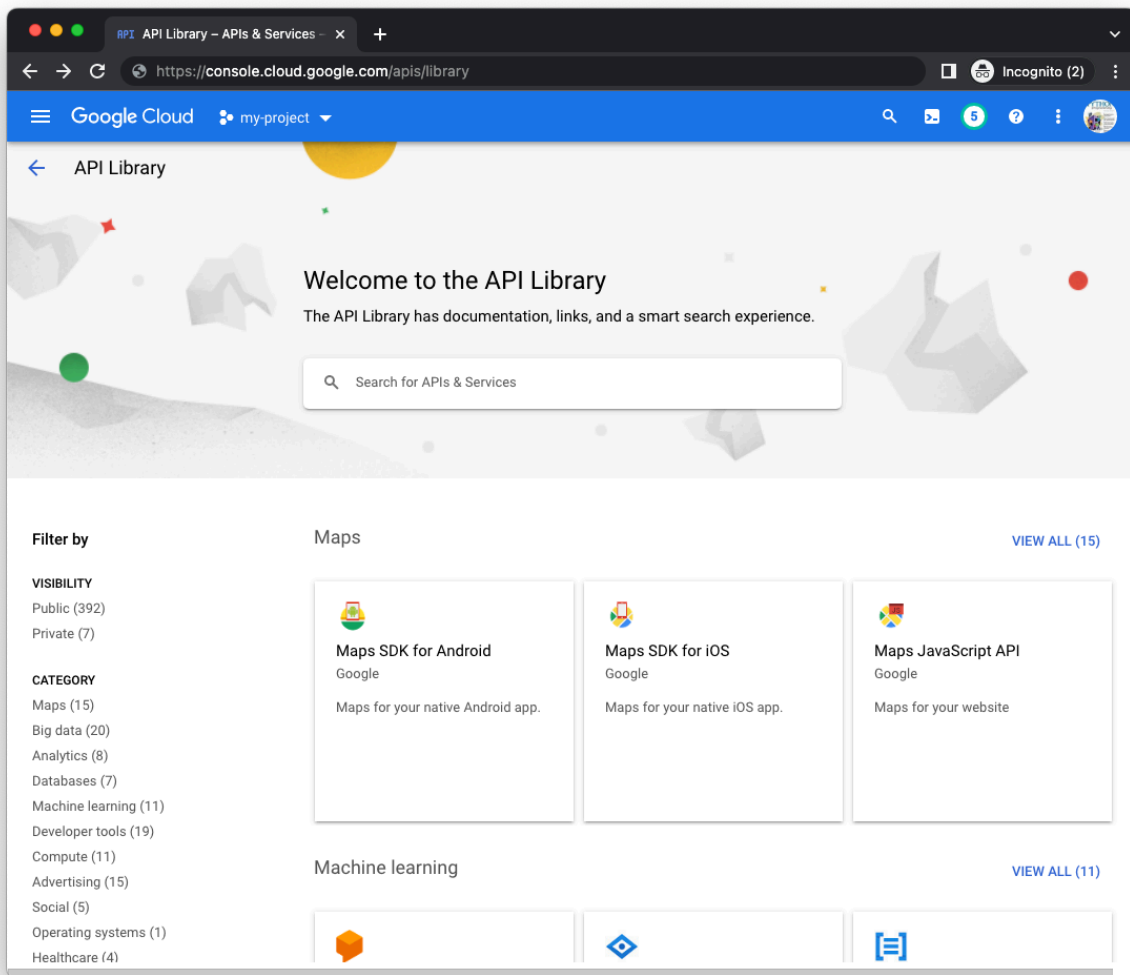
費用

- App Engine 和 Cloud Datastore 都有「[一律免費](#)」方案配額，只要不超出這些限制，完成本教學課程就不會產生費用。詳情請參閱 [App Engine 定價頁面](#)和 [Cloud Datastore 定價頁面](#)。
- 使用 Cloud Identity Platform 時，系統會根據每月活躍使用者人數 (MAU) 或驗證次數計費；每種用量模式都有「免費」版本。詳情請參閱[定價頁面](#)。此外，雖然 App Engine 和 Cloud Datastore 需要帳單，但只要不超過 [GCIP 的無儀器每日配額](#)，使用 GCIP 本身不需要啟用帳單，因此對於不涉及需要帳單的 Cloud API/服務的 Cloud 專案，請考慮使用 GCIP。
- 根據[定價頁面](#)，Cloud Resource Manager API 大多可免費使用。

使用者可以透過 [Cloud 控制台](#)或指令列 (透過 [Cloud SDK](#) 的 `gcloud` 指令) 啟用 Cloud API，視個人偏好而定。我們先從 Cloud Datastore 和 Cloud Resource Manager API 開始。

透過 Cloud 控制台

前往 Cloud Console 的 [API 管理員程式庫頁面](#) (適用於正確的專案)，然後使用搜尋列搜尋 API。



啟用下列 API：

- [Cloud Datastore](#)
- [Cloud Resource Manager](#)

分別找出並點選每個 API 的「啟用」按鈕，系統可能會提示您提供帳單資訊。舉例來說，以下是 Resource Manager API 的頁面：

 Google Cloud 





Cloud Resource Manager API

[Google Enterprise API](#)

Creates, reads, and updates metadata for Google Cloud Platform resource containers.

ENABLE

TRY THIS API 

OVERVIEW

DOCUMENTATION

啟用後 (通常需要幾秒鐘)，按鈕會變更為「管理」：

 Google Cloud 





Cloud Resource Manager API

[Google Enterprise API](#)

Creates, reads, and updates metadata for Google Cloud Platform resource containers.

MANAGE

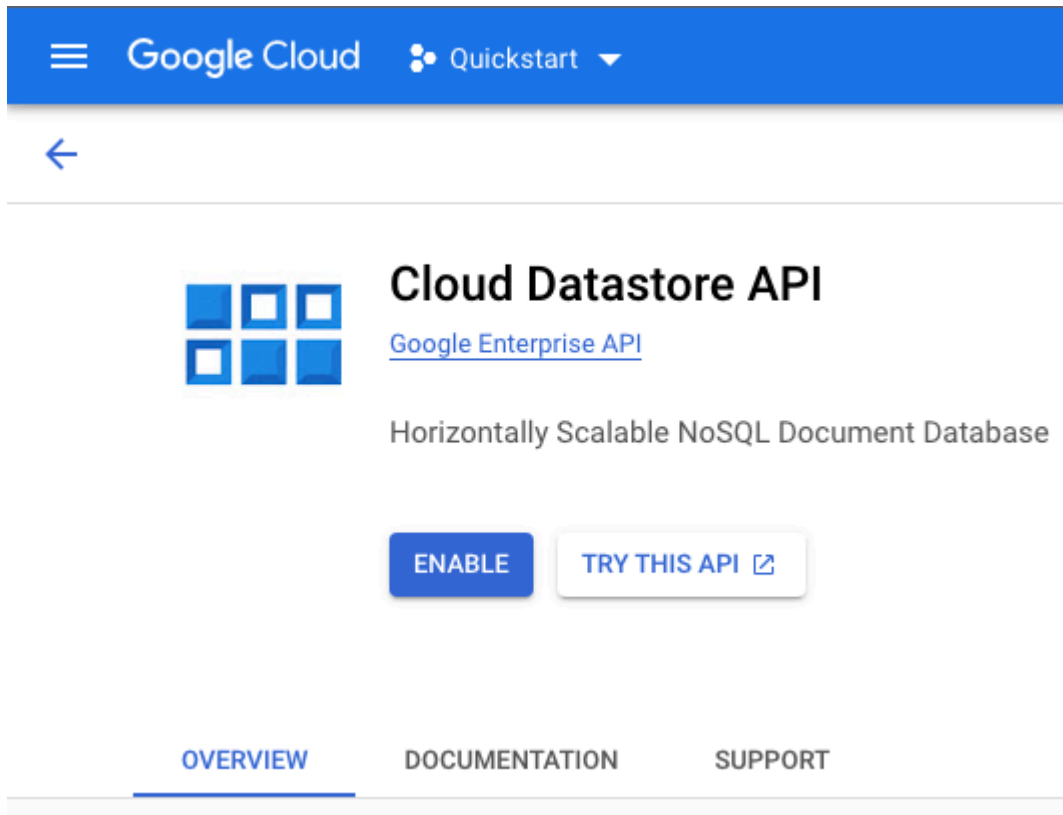
TRY THIS API 

 API Enabled

OVERVIEW

DOCUMENTATION

以相同方式啟用 Cloud Datastore：



使用指令列

雖然透過主控台啟用 API 具有視覺上的資訊性，但有些人偏好使用指令列。此外，您還能一次啟用任意數量的 API。發出這項指令來啟用 Cloud Datastore 和 Cloud Resource Manager API，並等待作業完成，如下所示：

```
$ gcloud services enable cloudresourcemanager.googleapis.com datastore.googleapis.com
Operation "operations/acat.p2-aaa-bbb-ccc-ddd-eee-ffffff" finished successfully.
```

系統可能會提示你輸入帳單資訊。

上述指令中使用的每個 API 的「URL」稱為 API 服務名稱，可在每個 API 的程式庫頁面底部找到。如要為自己的應用程式啟用其他 Cloud API，請在對應的 API 頁面中找到服務名稱。這項指令會列出可啟用 API 的所有服務名稱：

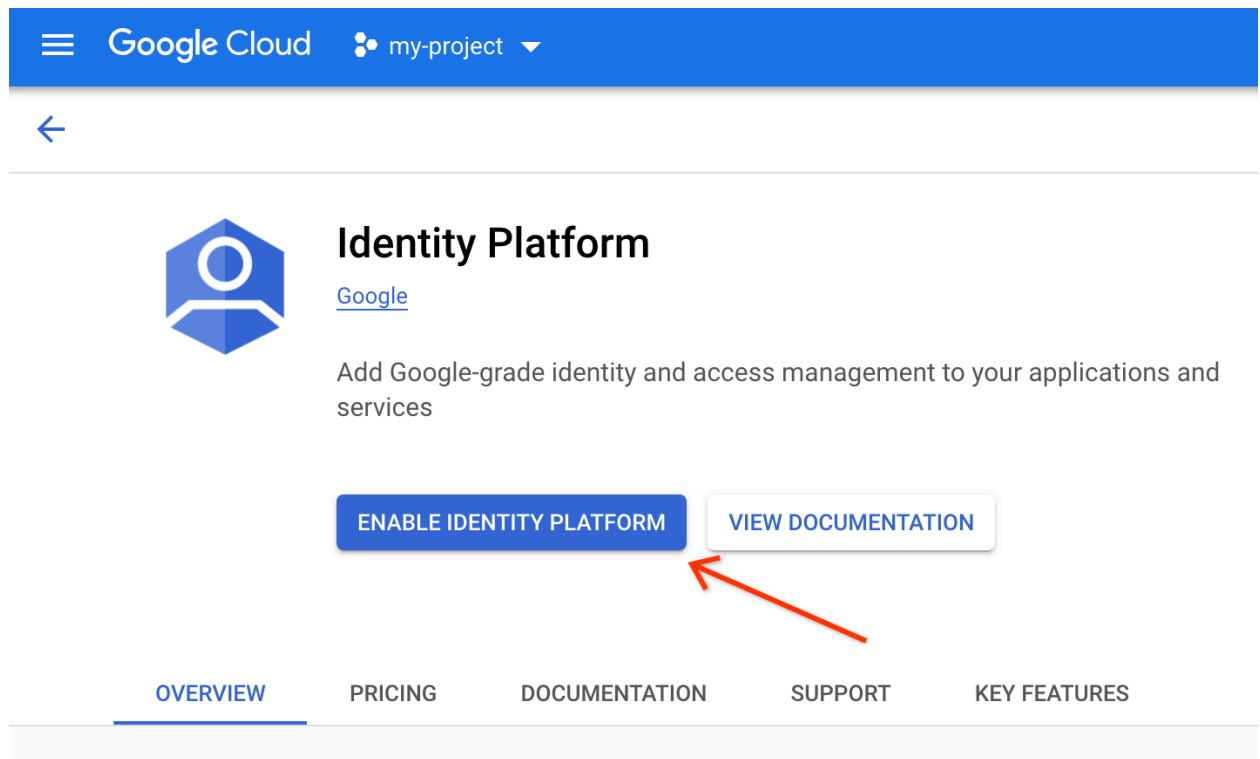
```
gcloud services list --available --filter="name:googleapis.com"。
```

無論是在 Cloud 控制台或指令列中，完成上述步驟後，我們的範例現在都能存取這些 API。接下來請啟用 Cloud Identity Platform，並進行必要的程式碼變更。

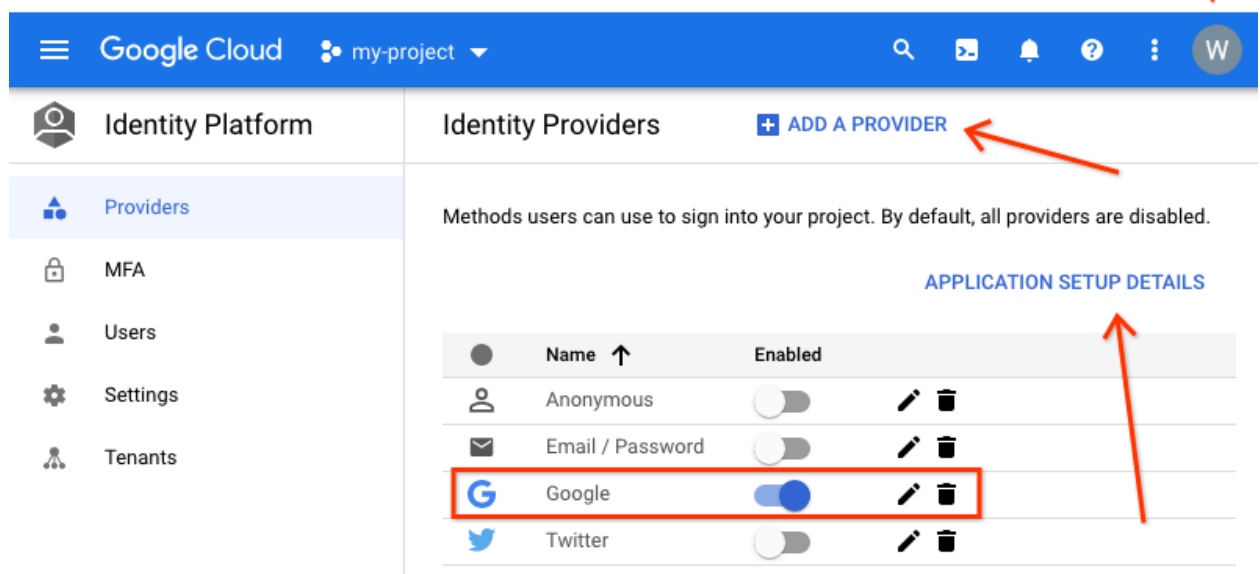
啟用及設定 Cloud Identity Platform (僅限 Cloud 控制台)

Cloud Identity Platform 是 [Marketplace 服務](#)，因為它會連線至 Google Cloud 外部的資源或依附於這類資源，例如 [Firebase Authentication](#)。目前只能透過 Cloud 控制台啟用 Marketplace 服務。步驟如下：

1. 前往 [Cloud Marketplace 的 Cloud Identity Platform 頁面](#)，然後按一下「啟用」按鈕。如果系統提示，請從 Firebase 驗證升級，這樣就能解鎖更多功能，例如背景一節中說明的那些功能。以下是 Marketplace 頁面，醒目顯示「啟用」按鈕：



2. 啟用 Identity Platform 後，系統可能會自動將您帶往「Identity Providers」(身分識別提供者) 頁面。如果沒有，請使用 [這個便利連結](#) 前往該頁面。



3. 啟用 Google 驗證供應器。如果尚未設定任何供應商，請按一下「新增供應商」，然後選取「Google」。返回這個畫面時，**Google** 項目應會啟用。在本教學課程中，我們只使用 Google 做為驗證提供者，以模擬 App Engine Users 服務，做為輕量型 Google 登入服務。您可以在自己的應用程式中啟用其他驗證供應商。
4. 選取並設定 Google 和其他所需的驗證供應商後，按一下「應用程式設定詳細資料」，然後在隨即顯示的對話方塊中，複製「網頁」分頁中 `config` 物件的 `apiKey` 和 `authDomain`，並將兩者儲存在安全的地方。為什麼不複製所有內容？這個對話方塊中的程式碼片段是硬式編碼，而且已過時，因此請只儲存最重要的部分，並在程式碼中使用這些部分，同時更頻繁地使用 Firebase Auth。複製值並儲存到安全的地方後，請按一下「關閉」按鈕，完成所有必要設定。

Google Cloud

my-project

Identity Platform

Providers

MFA

Users

Settings

Tenants

Configure your application

You can use the credentials below to add Identity Platform to your application, or select a getting started guide below. [Learn more.](#)

apiKey

ABCDEFDghA5UCmBeWzFNhpLatqBEYjUL1abcdef

authDomain

demo.firebaseapp.com

Getting started

WEBIOSANDROID

Copy and paste the snippet below at the bottom of your HTML, before other script tags. [Learn more](#)

<script src="https://www.gstatic.com/firebasejs/8.0/firebase.js"></script>

<script>

var config = {

apiKey: "ABCDEFDghA5UCmBeWzFNhpLatqBEYjUL1abcdef"

authDomain: "demo.firebaseapp.com",

};

firebase.initializeApp(config);

</script>

CLOSE

4. 更新設定

設定更新包括變更各種設定檔，以及在 Cloud Identity Platform 生態系統中建立 App Engine 等效項目。

appengine_config.py

- 如要升級至 Python 3，請刪除 `appengine_config.py`
- 如果您打算改用 Identity Platform，但仍使用 Python 2，請勿刪除該檔案。我們會在稍後進行 Python 2 回溯移植時更新。

requirements.txt

模組 20 的 `requirements.txt` 檔案只列出 [Flask](#)。針對模組 21，請新增下列套件：

- [Google 驗證](#)
- [Cloud NDB](#)
- [Cloud Resource Manager](#)
- [Firebase Admin SDK](#)

`requirements.txt` 的內容現在應如下所示：

```
flask
google-auth
google-cloud-ndb
google-cloud-resource-manager
firebase-admin
```

套件版本號碼

Python 3 的這個 `requirements.txt` 檔案沒有任何版本號碼，表示選取的是最新版本。如果發生任何不相容的情況，請按照標準做法，使用版本號碼鎖定應用程式的可用版本。

舉例來說，許多 Python 2 套件已遭淘汰，因此有時需要版本號碼，才能鎖定與 2.x 相容的套件。Python 2 `requirements.txt` 檔案中確實有版本號碼。我們將這個應用程式回溯移植到 Python 2 時，會進一步說明這項功能。另請參閱「Module 21a」資料夾。

app.yaml

- 升級至 Python 3 後，您就能簡化 `app.yaml` 檔案。移除所有內容 (執行階段指令除外)，並將執行階段指令設為目前支援的 Python 3 版本。這個範例目前使用 3.10 版。
- 如果繼續使用 Python 2，目前不必採取任何行動。

BEFORE:

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app
```

Module 20 範例應用程式沒有靜態檔案處理常式。如果您的應用程式會用到這些檔案，請保留這些檔案。您可以視需要移除所有指令碼處理常式，也可以保留這些處理常式做為參考，只要將處理常式變更為 `auto` 即可，如 [app.yaml 遷移指南](#) 所述。異動生效後，Python 3 的更新版 `app.yaml` 將簡化為：

修改後：

```
runtime: python310
```

其他設定更新

無論是繼續使用 Python 2 或移植到 Python 3，如果您的專案有 `lib` 資料夾，請刪除該資料夾。

5. 修改應用程式程式碼

本節介紹主要應用程式檔案 `main.py` 的更新，以 Cloud Identity Platform 取代 App Engine Users 服務。更新主要應用程式後，請更新網頁範本 `templates/index.html`。

更新匯入作業和初始化

請按照下列步驟更新匯入項目並初始化應用程式資源：

1. 在匯入項目中，將 App Engine NDB 替換為 Cloud NDB。
2. 除了 Cloud NDB，也請匯入 Cloud Resource Manager。
3. Identity Platform 以 Firebase 驗證為基礎，因此請匯入 Firebase Admin SDK。
4. Cloud API 需要使用 API 用戶端，因此請在初始化 Flask 後立即啟動 Cloud NDB。

雖然這裡匯入了 Cloud Resource Manager 套件，但我們會在應用程式初始化階段的稍後階段使用。以下是第 20 堂課的匯入和初始化作業，以及實作上述變更後各節的樣貌：

BEFORE:

```
from flask import Flask, render_template, request
from google.appengine.api import users
from google.appengine.ext import ndb

app = Flask(__name__)
```

修改後：

```
from flask import Flask, render_template, request
from google.auth import default
from google.cloud import ndb, resourcemanager
from firebase_admin import auth, initialize_app

# initialize Flask and Cloud NDB API client
app = Flask(__name__)
ds_client = ndb.Client()
```

App Engine 管理員使用者支援

如要將管理員使用者辨識功能新增至應用程式，需要新增兩個元件：

- `_get_gae_admins()` - 彙整一組管理員使用者；呼叫一次並儲存
- `is_admin()`：檢查登入的使用者是否為管理員使用者；在任何使用者登入時呼叫

公用函式 `_get_gae_admins()` 會呼叫 Resource Manager API，擷取目前的 [Cloud IAM allow-policy](#)。允許政策會定義並強制執行要將哪些角色授予哪些主體 (人類使用者、服務帳戶等)。設定內容包括：

- 擷取 Cloud 專案 ID (`PROJ_ID`)
- 建立 Resource Manager API 用戶端 (`rm_client`)
- 建立一組 (唯讀) App Engine 管理員角色 (`_TARGETS`)

Resource Manager 需要 Cloud 專案 ID，因此請匯入 `google.auth.default()` 並呼叫該函式來取得專案 ID。該呼叫包含的參數看起來像網址，但其實是 OAuth2 權限範圍。在雲端執行應用程式時 (例如在 Compute Engine VM 或 App Engine 應用程式上)，系統會提供具有廣泛權限的預設服務帳戶。為遵循最小權限最佳做法，建議您建立自己的使用者管理服務帳戶。

對於 API 呼叫，最好進一步將應用程式的範圍縮減至正常運作所需的最少程度。我們要發出的 Resource Manager API 呼叫是 `get_iam_policy()`，需要下列其中一個範圍才能運作：

- `https://www.googleapis.com/auth/cloud-platform`
- `https://www.googleapis.com/auth/cloud-platform.read-only`
- `https://www.googleapis.com/auth/cloudplatformprojects`
- `https://www.googleapis.com/auth/cloudplatformprojects.readonly`

範例應用程式只需要 `allow-policy` 的唯讀存取權。這項功能不會修改政策，也不需要存取整個專案。也就是說，應用程式不需要前三項權限。最後一個是必要項目，也是我們為範例應用程式實作的項目。

函式的主體會建立一組空白的管理員使用者 (`admins`)，透過 `get_iam_policy()` 擷取 `allow_policy`，並逐一檢查所有繫結，特別是 App Engine 管理員角色：

- `roles/viewer`
- `roles/editor`
- `roles/owner`
- `roles/appengine.appAdmin`

系統會針對找到的每個目標角色，彙整屬於該角色的使用者，並將他們加入管理員使用者整體組合。最後，系統會傳回所有找到並快取為常數 (`_ADMINS`) 的管理員使用者，做為這個 App Engine 執行個體的生命週期。我們很快就會看到該通電話。

在例項化 Cloud NDB API 用戶端 (`ds_client`) 的正下方，將下列 `_get_gae_admins()` 函式定義新增至 `main.py`：

```
def _get_gae_admins():
    'return set of App Engine admins'
    # setup constants for calling Cloud Resource Manager API
    __, PROJ_ID = default( # Application Default Credentials and project ID
        ['https://www.googleapis.com/auth/cloudplatformprojects.readonly'])
    rm_client = resourcemanager.ProjectsClient()
    _TARGETS = frozenset(( # App Engine admin roles
        'roles/viewer',
        'roles/editor',
        'roles/owner',
        'roles/appengine.appAdmin',
    ))

    # collate users who are members of at least one GAE admin role (_TARGETS)
    admins = set() # set of all App Engine admins
    allow_policy = rm_client.get_iam_policy(resource='projects/%s' % PROJ_ID)
    for b in allow_policy.bindings: # bindings in IAM allow-policy
        if b.role in _TARGETS: # only look at GAE admin roles
            admins.update(user.split(':', 1).pop() for user in b.members)
    return admins
```

使用者登入應用程式時，會發生下列情況：

1. 使用者登入 Firebase 後，系統會從網頁範本進行快速檢查。
2. 範本中的驗證狀態變更時，系統會對 `/is_admin` 進行 Ajax 樣式的 `fetch()` 呼叫，而下一個函式 `is_admin()` 則是處理常式。
3. Firebase ID 權杖會傳遞至 POST 主體中的 `is_admin()`，後者會從標頭中擷取權杖，並呼叫 Firebase Admin SDK 驗證權杖。如果是有效使用者，請擷取他們的電子郵件地址，並檢查是否為管理員使用者。
4. 然後，系統會將布林值結果傳回範本，並顯示 200 成功狀態。

在 `_get_gae_admins()` 後方新增 `is_admin()` 至 `main.py`：

```
@app.route('/is_admin', methods=['POST'])
def is_admin():
    'check if user (via their Firebase ID token) is GAE admin (POST) handler'
    id_token = request.headers.get('Authorization')
    email = auth.verify_id_token(id_token).get('email')
    return {'admin': email in _ADMINS}, 200
```

您必須複製這兩個函式的所有程式碼，才能重現 Users 服務提供的功能，尤其是 `is_current_user_admin()` 函式。與實作替代解決方案的模組 21 不同，模組 20 中的這個函式呼叫負責所有繁重的工作。好消息是，應用程式不再依附於僅限 App Engine 的服務，因此您可以將應用程式遷移至 Cloud Run 或其他服務。此外，您也可以將 `_TARGETS` 中切換至所需角色，為自己的應用程式變更「管理員使用者」的定義，但使用者服務會針對 App Engine 管理員角色進行硬式編碼。

設計考量：快取

當這個應用程式在任何 App Engine 執行個體上啟動時，系統會快取管理員使用者集。很可能新增了管理員使用者，導致快取失效。假設使用者不會經常成為管理員，而且在任何執行個體的生命週期內，這種情況發生的機率更低。如果您的應用程式並非如此，且這個清單會隨著高流量應用程式定期變更，您可能必須重新設計這個架構，並實作定期更新，以更新快取管理員使用者集。

初始化 Firebase Auth 並快取 App Engine 管理員使用者

我們可以在 Flask 應用程式初始化和 Cloud NDB API 用戶端建立的附近，初始化 Firebase Auth，但直到定義所有管理員程式碼 (也就是現在)，都沒有這個必要。同樣地，現在已定義 `_get_gae_admins()`，請呼叫該函式來快取管理員使用者清單。

在 `is_admin()` 的函式主體下方新增以下幾行：

```
# initialize Firebase and fetch set of App Engine admins
initialize_app()
_ADMINS = _get_gae_admins()
```

查看資料模型更新

visit 資料模型不會變更。若要存取 Datastore，必須明確使用 Cloud NDB API 用戶端內容管理工具 `ds_client.context()`。在程式碼中，這表示您會在 Python `with` 區塊內，將 Datastore 呼叫包裝在 `store_visit()` 和 `fetch_visits()` 中。本次更新與模組 2 相同。按照下列方式變更：

BEFORE:

```

class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor    = ndb.StringProperty()
    timestamp  = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    Visit(visitor='{0}: {1}'.format(remote_addr, user_agent)).put()

def fetch_visits(limit):
    'get most recent visits'
    return Visit.query().order(-Visit.timestamp).fetch(limit)

```

修改後：

```

class Visit(ndb.Model):
    'Visit entity registers visitor IP address & timestamp'
    visitor    = ndb.StringProperty()
    timestamp  = ndb.DateTimeProperty(auto_now_add=True)

def store_visit(remote_addr, user_agent):
    'create new Visit entity in Datastore'
    with ds_client.context():
        Visit(visitor='{0}: {1}'.format(remote_addr, user_agent)).put()

def fetch_visits(limit):
    'get most recent visits'
    with ds_client.context():
        return Visit.query().order(-Visit.timestamp).fetch(limit)

```

將使用者登入邏輯移至網頁範本

App Engine Users 服務屬於伺服器端，而 Firebase 驗證和 Cloud Identity Platform 則主要屬於用戶端。因此，Module 20 應用程式中的許多使用者管理程式碼都會移至 Module 21 網頁範本。

在 `main.py` 中，網頁環境會將五項重要資料傳遞至範本，前四項與使用者管理相關，且會因使用者是否登入而有所不同：

- `who` - 使用者的電子郵件地址 (如果已登入)，否則為 **user**
- `admin` - 如果登入使用者是管理員，則顯示「管理員」徽章
- `sign` : 顯示「登入」或「登出」按鈕
- `link` - 按一下按鈕即可登入或登出
- `visits` - 最近造訪的網站

BEFORE:

```

@app.route('/')
def root():
    'main application (GET) handler'
    store_visit(request.remote_addr, request.user_agent)
    visits = fetch_visits(10)

    # put together users context for web template
    user = users.get_current_user()
    context = { # logged in
        'who': user.nickname(),
        'admin': '(admin)' if users.is_current_user_admin() else '',
        'sign': 'Logout',
        'link': '/_ah/logout?continue=%s://%s/' % (
            request.environ['wsgi.url_scheme'],
            request.environ['HTTP_HOST'],
        ), # alternative to users.create_logout_url()
    } if user else { # not logged in
        'who': 'user',
        'admin': '',
        'sign': 'Login',
        'link': users.create_login_url('/'),
    }

    # add visits to context and render template
    context['visits'] = visits # display whether logged in or not
    return render_template('index.html', **context)

```

所有使用者管理作業都會移至網頁範本，因此我們只會留下造訪次數，讓主要處理常式回到第 1 模組應用程式中的狀態：

修改後：

```

@app.route('/')
def root():
    'main application (GET) handler'
    store_visit(request.remote_addr, request.user_agent)
    visits = fetch_visits(10)
    return render_template('index.html', visits=visits)

```

更新網頁範本

範本中會如何顯示上一節的所有更新？主要是將使用者管理功能從應用程式移至範本中執行的 Firebase Auth，並將所有移入 JavaScript 的程式碼部分移植。我們發現 `main.py` 大幅縮減，因此預期 `templates/index.html` 也會出現類似的成長。

BEFORE:

```
<!doctype html>
<html>
<head>
<title>VisitMe Example</title>
</head>
<body>
<p>
Welcome, {{ who }} <code>{{ admin }}</code>
<button id="logbtn">{{ sign }}</button>
</p><hr>

<h1>VisitMe example</h1>
<h3>Last 10 visits</h3>
<ul>
{% for visit in visits %}
  <li>{{ visit.timestamp.ctime() }} from {{ visit.visitor }}</li>
{% endfor %}
</ul>

<script>
document.getElementById("logbtn").onclick = () => {
  window.location.href = '{{ link }}';
};
</script>
</body>
</html>
```

將整個網頁範本替換為下列內容：

修改後：

```

<!doctype html>
<html>
<head>
<title>VisitMe Example</title>

<script type="module">
// import Firebase module attributes
import {
    initializeApp
} from "https://www.gstatic.com/firebasejs/9.10.0/firebase-app.js";
import {
    GoogleAuthProvider,
    getAuth,
    onAuthStateChanged,
    signInWithPopup,
    signOut
} from "https://www.gstatic.com/firebasejs/9.10.0/firebase-auth.js";

// Firebase config:
// 1a. Go to: console.cloud.google.com/customer-identity/providers
// 1b. May be prompted to enable GCIP and upgrade from Firebase
// 2. Click: "Application Setup Details" button
// 3. Copy: 'apiKey' and 'authDomain' from 'config' variable
var firebaseConfig = {
    apiKey: "YOUR_API_KEY",
    authDomain: "YOUR_AUTH_DOMAIN",
};

// initialize Firebase app & auth components
initializeApp(firebaseConfig);
var auth = getAuth();
var provider = new GoogleAuthProvider();
//provider.setCustomParameters({prompt: 'select_account'});

// define login and logout button functions
function login() {
    signInWithPopup(auth, provider);
};

function logout() {
    signOut(auth);
};

// check if admin & switch to logout button on login; reset everything on logout
onAuthStateChanged(auth, async (user) => {
    if (user && user != null) {
        var email = user.email;
        who.innerHTML = email;
        logbtn.onclick = logout;
        logbtn.innerHTML = "Logout";
        var idToken = await user.getIdToken();
        var rsp = await fetch("/is_admin", {

```

```

        method: "POST",
        headers: {Authorization: idToken}
    });
    var data = await rsp.json();
    if (data.admin) {
        admin.style.display = "inline";
    }
} else {
    who.innerHTML = "user";
    admin.style.display = "none";
    logbtn.onclick = login;
    logbtn.innerHTML = "Login";
}
});
</script>
</head>

<body>
<p>
Welcome, <span id="who"></span> <span id="admin"><code>(admin)</code></span>
<button id="logbtn"></button>
</p><hr>

<h1>VisitMe example</h1>
<h3>Last 10 visits</h3>
<ul>
{% for visit in visits %}
    <li>{{ visit.timestamp.ctime() }} from {{ visit.visitor }}</li>
{% endfor %}
</ul>

<script>
var who    = document.getElementById("who");
var admin  = document.getElementById("admin");
var logbtn = document.getElementById("logbtn");
</script>
</body>
</html>

```

這個 HTML 內文包含許多元件，因此我們將逐一說明。

Firestore 匯入

在 HTML 文件的標題中，匯入所需的 Firestore 元件。Firestore 元件現在會分成多個模組，以提升效率。初始化 Firestore 的程式碼是從主要的 Firestore 應用程式模組匯入，而管理 Firestore 驗證、Google 做為驗證供應商、登入和登出，以及驗證狀態變更「回呼」的函式，全都是從 Firestore Auth 模組匯入：

```
<!doctype html>
<html>
<head>
<title>VisitMe Example</title>

<script type="module">
// import Firebase module attributes
import {
  initializeApp
} from "https://www.gstatic.com/firebasejs/9.10.0/firebase-app.js";
import {
  GoogleAuthProvider,
  getAuth,
  onAuthStateChanged,
  signInWithPopup,
  signOut
} from "https://www.gstatic.com/firebasejs/9.10.0/firebase-auth.js";
```

Firestore 設定

在本教學課程的 Identity Platform 設定部分，您先前已從「Application Setup Details」(應用程式設定詳細資料) 對話方塊儲存 `apiKey` 和 `authDomain`。在下一個部分中，將這些值新增至 `firebaseConfig` 變數。留言中提供詳細操作說明的連結：

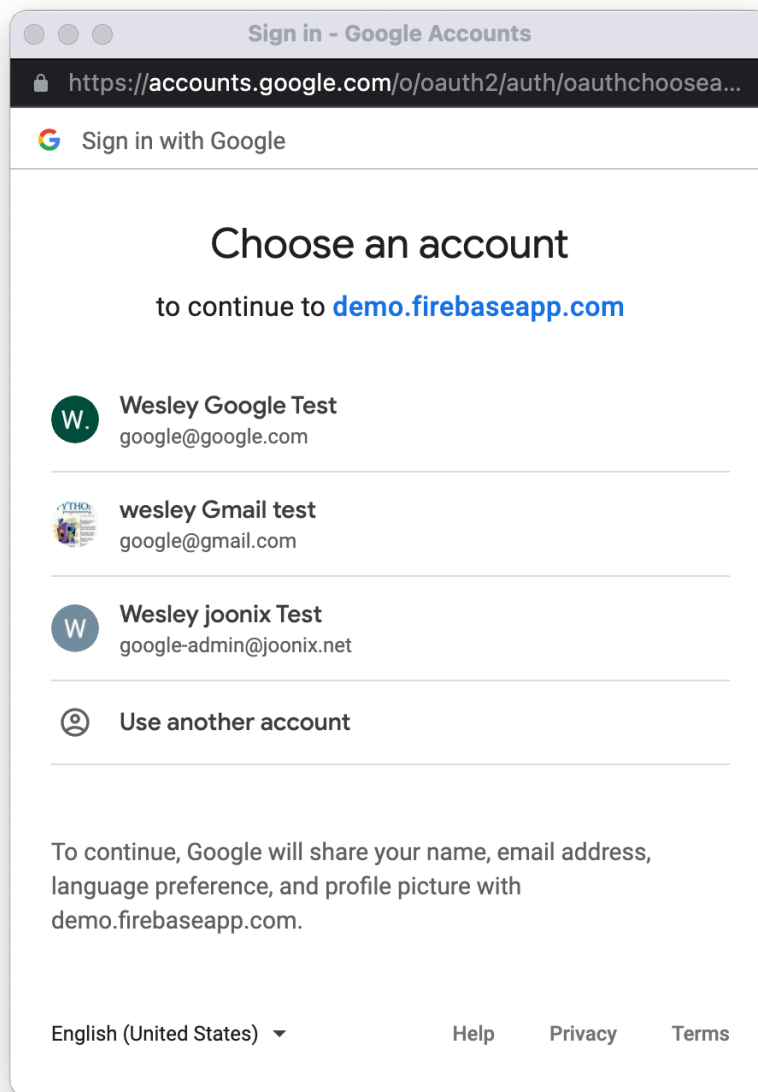
```
// Firebase config:
// 1a. Go to: console.cloud.google.com/customer-identity/providers
// 1b. May be prompted to enable GCIP and upgrade from Firebase
// 2. Click: "Application Setup Details" button
// 3. Copy: 'apiKey' and 'authDomain' from 'config' variable
var firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_AUTH_DOMAIN",
};
```

Firestore 初始化

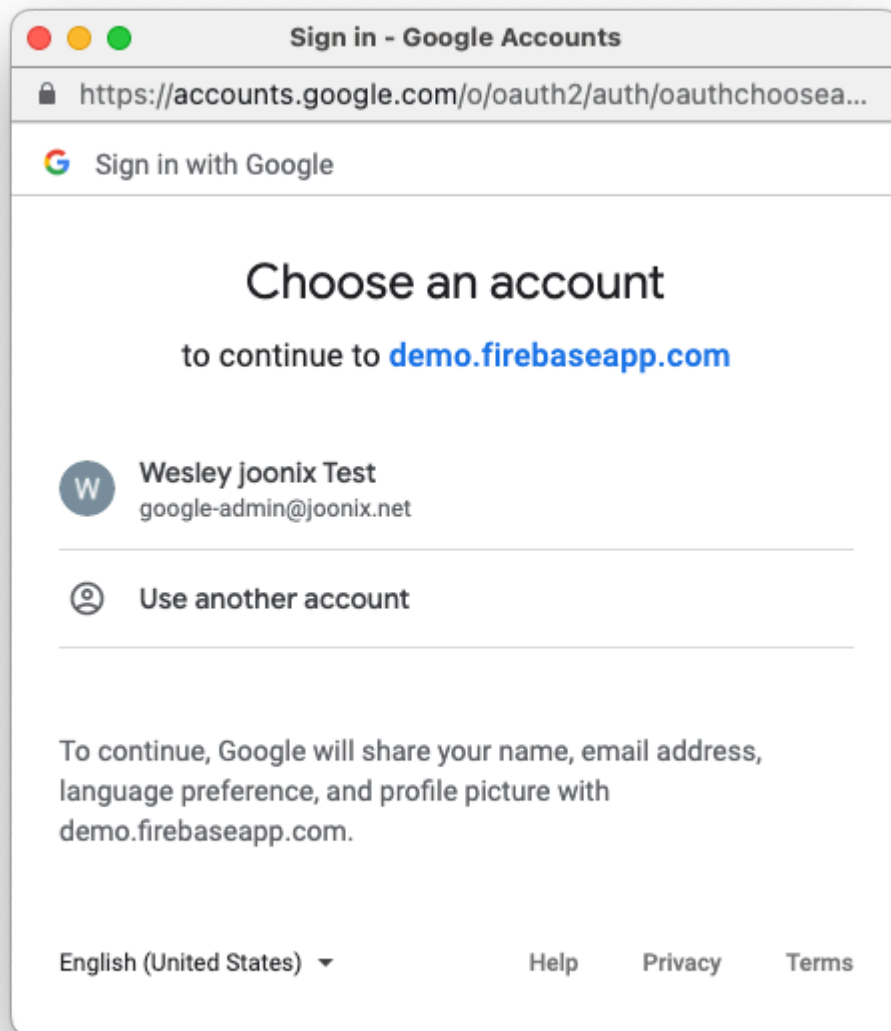
下一節會使用這項設定資訊初始化 Firestore。

```
// initialize Firebase app & auth components
initializeApp(firebaseConfig);
var auth = getAuth();
var provider = new GoogleAuthProvider();
//provider.setCustomParameters({prompt: 'select_account'});
```

這會設定使用 Google 做為驗證提供者的功能，並提供註解掉的選項，即使瀏覽器工作階段中只註冊了一個 Google 帳戶，也會顯示帳戶選取器。換句話說，如果您有多個帳戶，系統會如預期顯示「帳戶選擇器」：



不過，如果工作階段中只有一位使用者，登入程序會自動完成，不需要使用者互動。(彈出式視窗會顯示然後消失)。如要強制為一位使用者顯示帳戶選擇器對話方塊 (而非立即登入應用程式)，請取消註解自訂參數行。啟用後，即使是單一使用者登入，系統也會顯示帳戶選擇器：



登入和登出函式

接下來的程式碼行會組成登入或登出按鈕點擊的函式：

```
// define login and logout button functions
function login() {
  signInWithPopup(auth, provider);
};

function logout() {
  signOut(auth);
};
```

登入和登出動作

這個 `<script>` 區塊的最後一個主要部分，是針對每次驗證狀態變更 (登入或登出) 呼叫的函式。

```
// check if admin & switch to logout button on login; reset everything on logout
onAuthStateChanged(auth, async (user) => {
  if (user && user != null) {
    var email = user.email;
    who.innerHTML = email;
    logbtn.onclick = logout;
    logbtn.innerHTML = "Logout";
    var idToken = await user.getIdToken();
    var rsp = await fetch("/is_admin", {
      method: "POST",
      headers: {Authorization: idToken}
    });
    var data = await rsp.json();
    if (data.admin) {
      admin.style.display = "inline";
    }
  } else {
    who.innerHTML = "user";
    admin.style.display = "none";
    logbtn.onclick = login;
    logbtn.innerHTML = "Login";
  }
});
</script>
</head>
```

第 20 節中的程式碼會判斷要傳送「使用者已登入」範本內容，還是「使用者已登出」內容，並在此處轉換。如果使用者成功登入，頂端的條件會導致 `true`，並觸發下列動作：

1. 系統會顯示使用者的電子郵件地址。
2. 「登入」按鈕會變更為「登出」。
3. 系統會對 `/is_admin` 進行 Ajax 樣式的呼叫，判斷是否要顯示 `(admin)` 管理員使用者徽章。

使用者登出時，系統會執行 `else` 子句，重設所有使用者資訊：

1. 使用者名稱已設為「user」
2. 移除任何管理員徽章
3. 「登出」按鈕改回「登入」

範本變數

標頭區段結束後，主體會以範本變數開頭，這些變數會視需要替換為 HTML 元素：

1. 顯示的使用者名稱
2. `(admin)` 管理員徽章 (如適用)
3. 「登入」或「登出」按鈕

```
<body>
<p>
Welcome, <span id="who"></span> <span id="admin"><code>(admin)</code></span>
<button id="logbtn"></button>
</p><hr>
```

最近造訪次數和 HTML 元素變數

最近一次的造訪程式碼不會變更，而最後一個 `<script>` 區塊會為登入和登出時變更的 HTML 元素設定變數，如上所示：

```
<h1>VisitMe example</h1>
<h3>Last 10 visits</h3>
<ul>
{% for visit in visits %}
    <li>{{ visit.timestamp.ctime() }} from {{ visit.visitor }}</li>
{% endfor %}
</ul>

<script>
var who    = document.getElementById("who");
var admin  = document.getElementById("admin");
var logbtn = document.getElementById("logbtn");
</script>
</body>
</html>
```

至此，您已完成應用程式和網頁範本中所需的所有變更，可從 App Engine NDB 和 Users API 切換至 Cloud NDB 和 Identity Platform，並升級至 Python 3。恭喜！您已完成第 21 個模組的範例應用程式！您可以在 [Module 21b repo](#) 資料夾中查看我們的版本。

程式碼研究室的下一部分為選用 (*) 內容，僅適用於應用程式必須保留在 Python 2 的使用者，引導您完成必要步驟，建立可運作的 Python 2 模組 21 應用程式。

6. *Python 2 回溯移植

這個選用章節適用於要執行 Identity Platform 遷移作業，但必須繼續使用 Python 2 執行階段的開發人員。如果這不是您關心的問題，請略過這個部分。

如要建立 Module 21 應用程式的 Python 2 運作版本，您需要：

1. **執行階段需求**：支援 Python 2 的設定檔，以及主要應用程式中為避免 Python 3 不相容問題而必須進行的變更
2. **程式庫的微小變更**：在將部分必要功能新增至 Resource Manager 用戶端程式庫之前，Python 2 已遭淘汰。因此，您需要透過其他方式存取遺失的功能。

現在就開始執行這些步驟，首先是設定。

還原 appengine_config.py

在本教學課程稍早，我們引導您刪除 `appengine_config.py`，因為 Python 3 App Engine 執行階段不會使用這個檔案。如果是 Python 2，不僅必須保留，還需要更新模組 20 `appengine_config.py`，以支援使用內建的第三方程式庫，也就是 `grpcio` 和 `setuptools`。只要 App Engine 應用程式使用 Cloud NDB 和 Cloud Resource Manager 等 Cloud 用戶端程式庫，就必須安裝這些套件。

您稍後會將這些套件新增至 `app.yaml`，但如要讓應用程式存取這些套件，必須呼叫 `setuptools` 中的 `pkg_resources.working_set.add_entry()` 函式。這樣一來，安裝在 `lib` 資料夾中的複製 (自行組合或供應商提供) 第三方程式庫，就能與內建程式庫通訊。

如要套用這些變更，請在 `appengine_config.py` 檔案中進行下列更新：

BEFORE:

```
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
```

單靠這段程式碼，還不足以支援使用 `setuptools` 和 `grpcio`。還需要幾行程式碼，因此請更新 `appengine_config.py`，使其如下所示：

修改後：

```
import pkg_resources
from google.appengine.ext import vendor

# Set PATH to your libraries folder.
PATH = 'lib'
# Add libraries installed in the PATH folder.
vendor.add(PATH)
# Add libraries to pkg_resources working set to find the distribution.
pkg_resources.working_set.add_entry(PATH)
```

如要進一步瞭解支援 Cloud 用戶端程式庫所需的變更，請參閱[遷移套裝服務說明文件](#)。

app.yaml

與 `appengine_config.py` 類似，`app.yaml` 檔案也必須還原為支援 Python 2 的版本。我們先從原始的第 20 號模組 `app.yaml` 開始：

BEFORE:

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app
```

除了先前提及的 `setuptools` 和 `grpcio` 之外，還有一個依附元件 (與 Identity Platform 遷移作業沒有明確關聯)，需要使用 [Cloud Storage 用戶端程式庫](#)，而該程式庫需要另一個內建的第三方套件 `ssl`。在新的 `libraries` 區段中新增這三個項目，並選取這些套件的「最新」可用版本，然後按一下 `app.yaml`：

修改後：

```
runtime: python27
threadsafe: yes
api_version: 1

handlers:
- url: /*
  script: main.app

libraries:
- name: grpcio
  version: latest
- name: setuptools
  version: latest
- name: ssl
  version: latest
```

requirements.txt

在第 21 個模組中，我們新增了 Python 3 `requirements.txt` 的 [Google Auth](#)、[Cloud NDB](#)、[Cloud Resource Manager](#) 和 [Firebase Admin SDK](#)。Python 2 的情況較為複雜：

- Resource Manager API 提供範例應用程式所需的允許政策功能。可惜的是，[Cloud Resource Manager 用戶端程式庫](#) 的最終 [Python 2 版本](#) 尚未支援這項功能。(僅適用於 Python 3 版本)。
- 因此，您必須透過 API 存取這項功能。解決方法是使用較低層級的 [Google API 用戶端程式庫](#) 與 API 通訊。如要改用這個用戶端程式庫，請將 `google-cloud-resource-manager` 換成較低層級的 `google-api-python-client` 套件。
- 由於 [Python 2 已停止支援](#)，支援模組 21 的依附元件圖需要將特定套件鎖定在特定版本。即使 Python 3 `app.yaml` 中未指定某些套件，也必須標註。

BEFORE:

```
flask
```

從第 20 個模組 `requirements.txt` 開始，請更新至下列項目，才能使用第 21 個模組的應用程式：

修改後：

```
grpcio==1.0.0
protobuf<3.18.0
six>=1.13.0
flask
google-gax<0.13.0
google-api-core==1.31.1
google-api-python-client<=1.11.0
google-auth<2.0dev
google-cloud-datastore==1.15.3
google-cloud-firestore==1.9.0
google-cloud-ndb
google-cloud-pubsub==1.7.0
firebase-admin
```

隨著依附元件變更，存放區中的套件和版本號碼也會更新，但就撰寫本文時的運作應用程式而言，這個 `app.yaml` 就已足夠。

其他設定更新

如果您尚未刪除本程式碼研究室稍早的 `lib` 資料夾，請立即刪除。使用新版 `requirements.txt` 發出這個常見指令，將這些需求項目安裝到 `lib` 中：

```
pip install -t lib -r requirements.txt # or pip2
```

如果開發系統同時安裝 Python 2 和 3，可能需要使用 `pip2`，而非 `pip`。

修改應用程式程式碼

幸好，大部分的必要變更都在設定檔中。應用程式程式碼只需要進行小幅更新，改用較低層級的 Google API 用戶端程式庫，而非 Resource Manager 用戶端程式庫來存取 API。 `templates/index.html` 網頁範本不需要更新。

更新匯入項目和初始化作業

將 Resource Manager 用戶端程式庫 (`google.cloud.resourcemanager`) 替換為 Google API 用戶端程式庫 (`googleapiclient.discovery`)，如下所示：

BEFORE:

```
from flask import Flask, render_template, request
from google.auth import default
from google.cloud import ndb, resourcemanager
from firebase_admin import auth, initialize_app
```

修改後：

```
from flask import Flask, render_template, request
from google.auth import default
from google.cloud import ndb
from googleapiclient import discovery
from firebase_admin import auth, initialize_app
```

App Engine 管理員使用者支援

您需要在 `_get_gae_admins()` 中進行幾項變更，才能支援使用低階用戶端程式庫。我們先討論變更內容，然後提供所有更新程式碼。

Python 2 程式碼必須使用 `google.auth.default()` 傳回的憑證和專案 ID。Python 3 不會使用憑證，因此憑證已指派給一般底線 (`_`) 虛擬變數。由於 Python 2 版本需要這個函式，請將底線變更為 `CREDS`。此外，您會建立 API 服務端點 (概念類似於 API 用戶端)，而不是建立 Resource Manager API 用戶端，因此我們保留相同的變數名稱 (`rm_client`)。不同之處在於，例項化服務端點需要憑證 (`CREDS`)。

這些變更會反映在下列程式碼中：

BEFORE:

```
_, PROJ_ID = default( # Application Default Credentials and project ID
    ['https://www.googleapis.com/auth/cloudplatformprojects.readonly'])
rm_client = resourcemanager.ProjectsClient()
```

修改後：

```
CREDS, PROJ_ID = default( # Application Default Credentials and project ID
    ['https://www.googleapis.com/auth/cloud-platform'])
rm_client = discovery.build('cloudresourcemanager', 'v1', credentials=CREDS)
```

另一個差異是，Resource Manager 用戶端程式庫會傳回使用點屬性標記的 `allow-policy` 物件，而低階用戶端程式庫會傳回使用方括號 (`[]`) 的 Python 字典，例如 Resource Manager 用戶端程式庫使用 `binding.role`，而低階程式庫使用 `binding['role']`。前者也使用「底線分隔」名稱，而較低層級的程式庫偏好使用「駝峰式大小寫」名稱，且傳遞 API 參數的方式略有不同。

以下列出這些使用差異：

BEFORE:

```
allow_policy = rm_client.get_iam_policy(resource='projects/%s' % PROJ_ID)
for b in allow_policy.bindings: # bindings in IAM allow-policy
    if b.role in _TARGETS: # only look at GAE admin roles
        admins.update(user.split(':', 1).pop() for user in b.members)
```

修改後：

```
allow_policy = rm_client.projects().getIamPolicy(resource=PROJ_ID).execute()
for b in allow_policy['bindings']: # bindings in IAM allow-policy
    if b['role'] in _TARGETS: # only look at GAE admin roles
        admins.update(user.split(':', 1).pop() for user in b['members'])
```

將所有這些變更整合在一起，以這個對應的 Python 2 版本取代 Python 3 `_get_gae_admins()`：

```
def _get_gae_admins():
    'return set of App Engine admins'
    # setup constants for calling Cloud Resource Manager API
    CREDS, PROJ_ID = default( # Application Default Credentials and project ID
        ['https://www.googleapis.com/auth/cloud-platform'])
    rm_client = discovery.build('cloudresourcemanager', 'v1', credentials=CREDS)
    _TARGETS = frozenset(( # App Engine admin roles
        'roles/viewer',
        'roles/editor',
        'roles/owner',
        'roles/appengine.appAdmin',
    ))

    # collate users who are members of at least one GAE admin role (_TARGETS)
    admins = set() # set of all App Engine admins
    allow_policy = rm_client.projects().getIamPolicy(resource=PROJ_ID).execute()
    for b in allow_policy['bindings']: # bindings in IAM allow-policy
        if b['role'] in _TARGETS: # only look at GAE admin roles
            admins.update(user.split(':', 1).pop() for user in b['members'])
    return admins
```

`is_admin()` 函式不需要任何更新，因為它依賴的 `_get_gae_admins()` 函式已更新。

至此，我們已完成將 Python 3 模組 21 應用程式回溯移植到 Python 2 的必要變更。恭喜！您已完成更新後的第 21 個模組範例應用程式！您可以在[模組 21a 存放區資料夾](#)中找到所有程式碼。

7. 摘要/清除

程式碼研究室的最後一個步驟是確保執行這個應用程式的主體(使用者或服務帳戶) 具備適當權限，然後部署應用程式，確認應用程式運作正常，且變更會反映在輸出內容中。

可讀取 IAM 允許政策

我們先前介紹了四個角色，您必須擁有這些角色才能成為 App Engine 管理員使用者，但現在還有第五個角色需要瞭解：

- `roles/viewer`
- `roles/editor`
- `roles/owner`
- `roles/appengine.appAdmin`
- `roles/resourcemanager.projectIamAdmin` (適用於存取 IAM 允許政策的主體)

`roles/resourcemanager.projectIamAdmin` 角色可讓主體判斷使用者是否屬於任何 App Engine 管理員角色。如果沒有 `roles/resourcemanager.projectIamAdmin` 的成員資格，呼叫 Cloud Resource Manager API 來取得允許政策就會失敗。

您不需要在此採取任何明確行動，因為應用程式會在 [App Engine 的預設服務帳戶](#) 下執行，系統會自動授予該帳戶這個角色的成員資格。即使在開發階段使用預設服務帳戶，我們仍強烈建議您建立並使用 [使用者自行管理的服務帳戶](#)，並只授予應用程式正常運作所需的最低權限。如要授予這類服務帳戶成員資格，請執行下列指令：

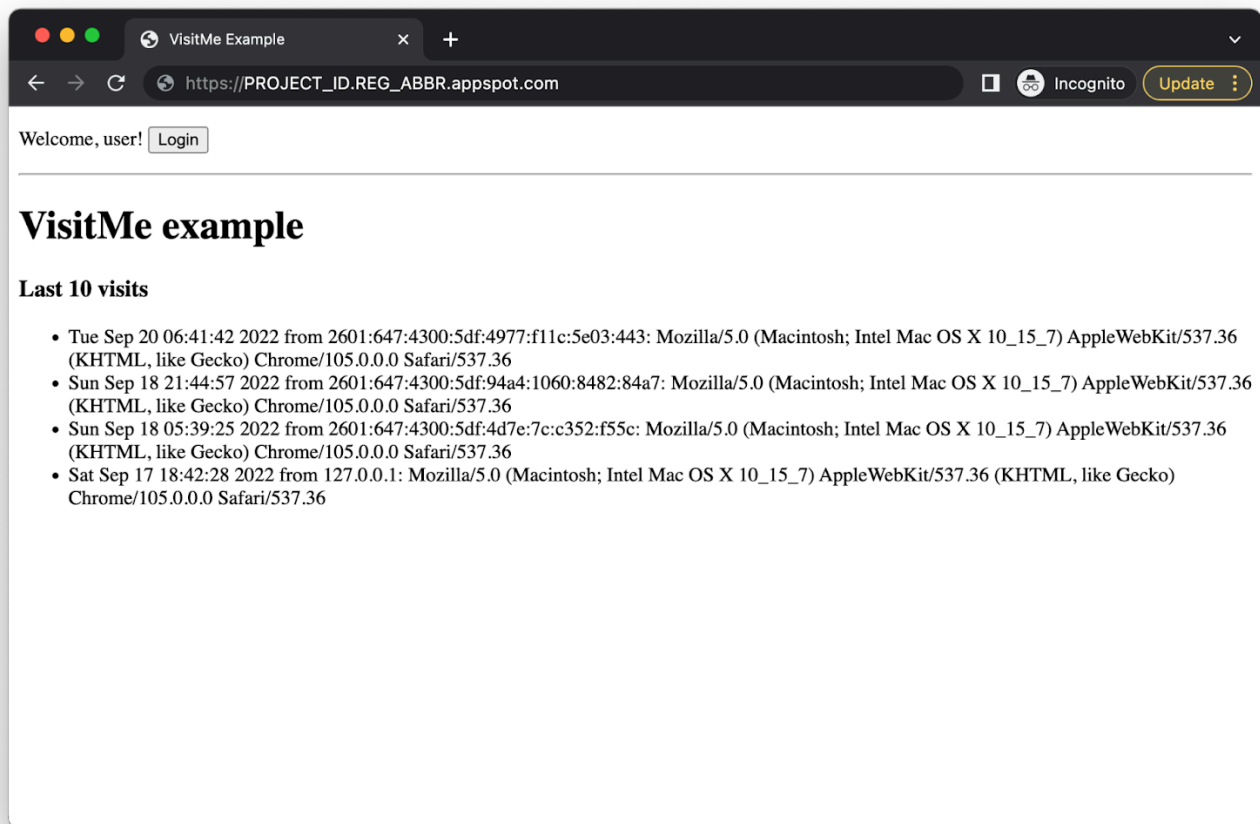
```
$ gcloud projects add-iam-policy-binding PROJ_ID --  
member="serviceAccount:USR_MGD_SVC_ACCT@PROJ_ID.iam.gserviceaccount.com" --  
role=roles/resourcemanager.projectIamAdmin
```

`PROJ_ID` 是 Cloud 專案 ID，`USR_MGD_SVC_ACCT@PROJ_ID.iam.gserviceaccount.com` 則是您為應用程式建立的使用者管理服務帳戶。這項指令會輸出專案的最新 IAM 政策，您可以在其中確認服務帳戶是否為

`roles/resourcemanager.projectIamAdmin` 的成員。詳情請參閱 [參考說明文件](#)。再次提醒，您不需要在本程式碼研究室中發出該指令，但請儲存這項資訊，以供日後更新自己的應用程式時參考。

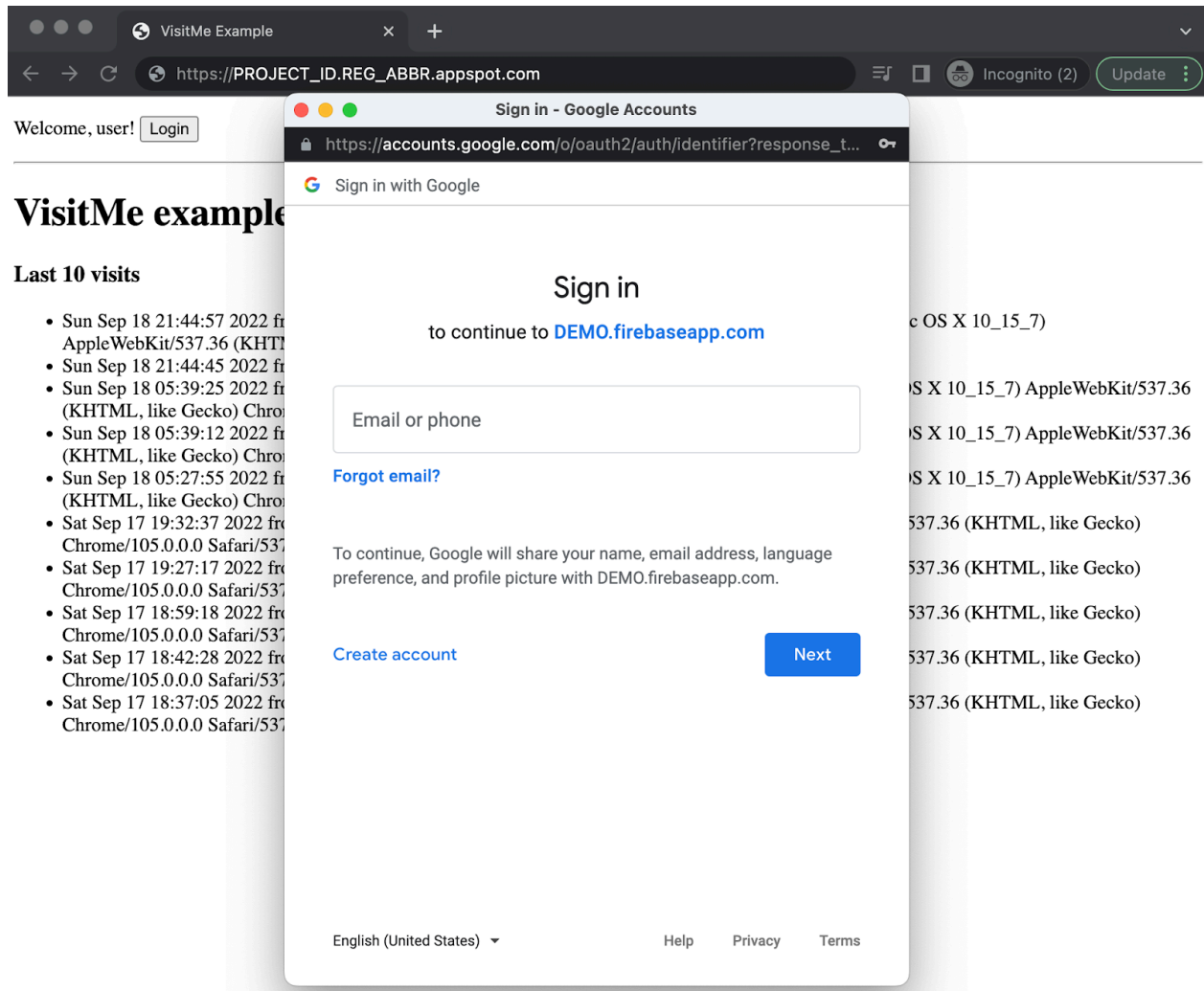
部署及驗證應用程式

使用標準 `gcloud app deploy` 指令將應用程式上傳至雲端。部署完成後，您應該會發現功能與第 20 堂課的應用程式幾乎完全相同，但您已成功將 App Engine Users 服務替換為 Cloud Identity Platform (和 Firebase Auth)，用於使用者管理：



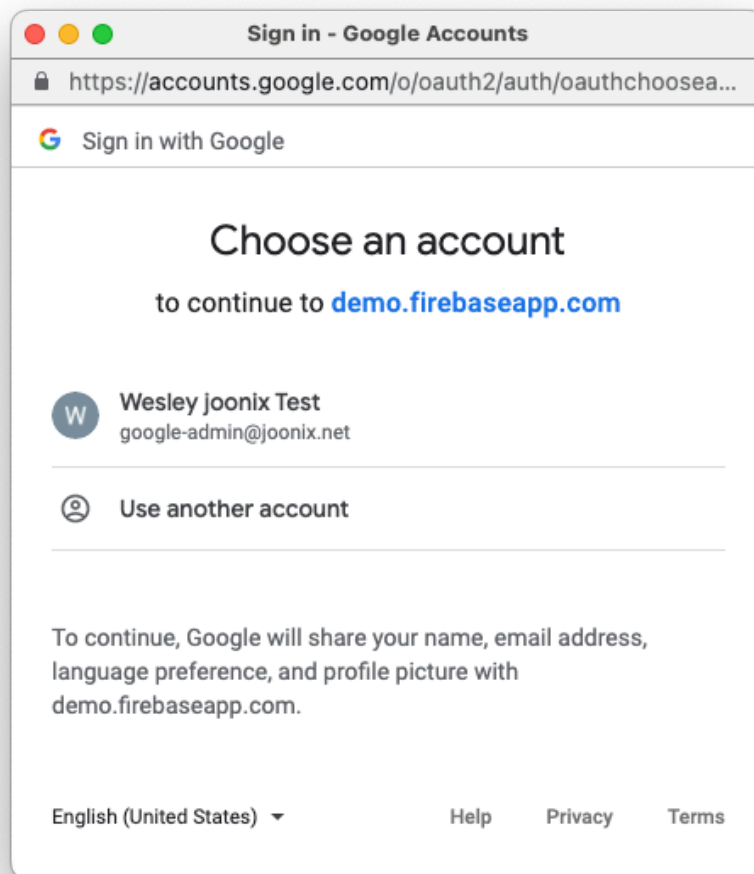
與第 20 堂課相比，您會發現點選「登入」後，系統會顯示彈出式視窗，而不是重新導向，如下方部分螢幕截圖所示。不過，與第 20 節相同，行為會因瀏覽器已註冊的 Google 帳戶數量而略有不同。

如果瀏覽器沒有已註冊的使用者，或是只有一位使用者但尚未登入，系統會顯示一般 Google 登入彈出式視窗：



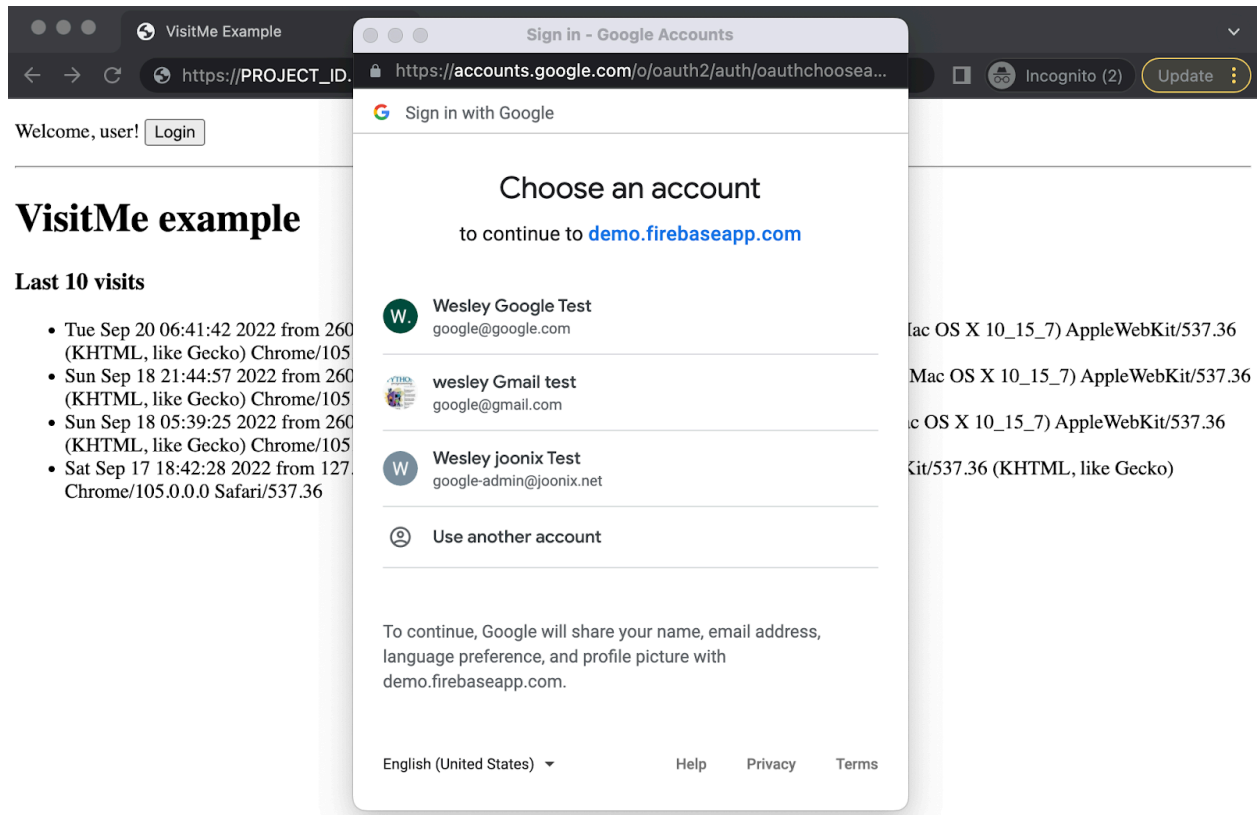
如果單一使用者已在瀏覽器中註冊，但從其他地方登入，系統不會顯示對話方塊 (或對話方塊會彈出並立即關閉)，且應用程式會進入登入狀態 (顯示使用者電子郵件地址和「登出」按鈕)。

即使只有一位使用者，部分開發人員可能還是想提供帳戶選擇器：

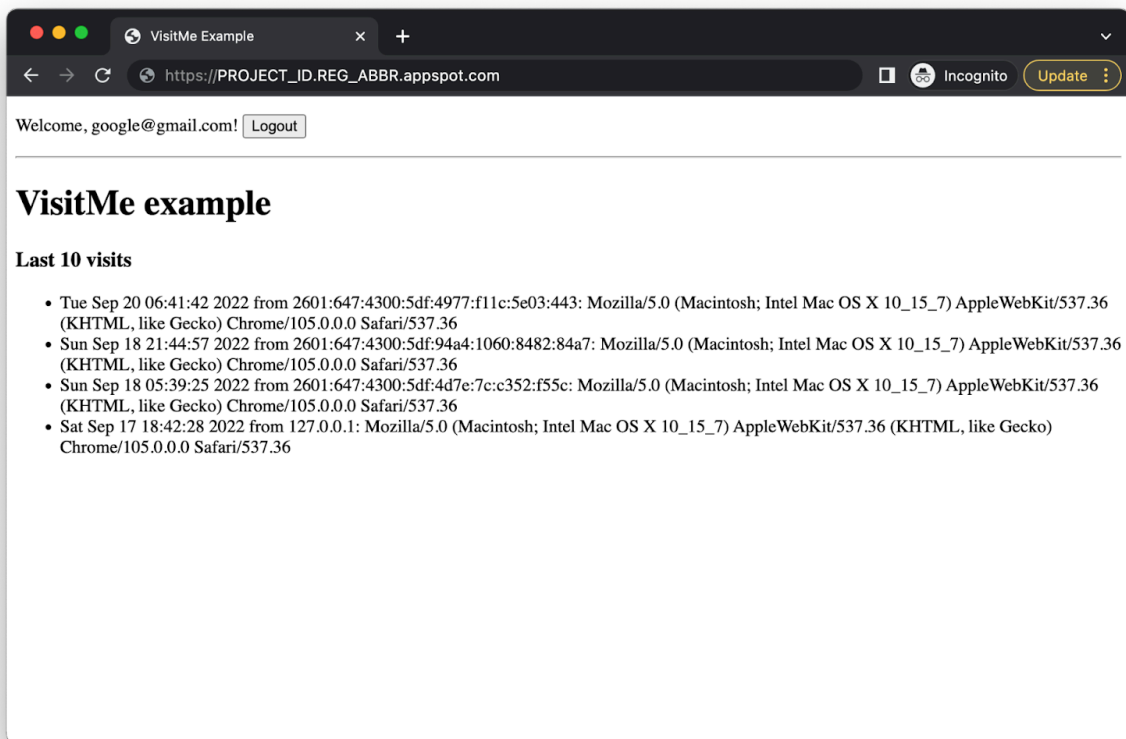


如要實作這項功能，請按照先前的說明，取消註解網頁範本中的 `provider.setCustomParameters({prompt: 'select_account'})`; 行。

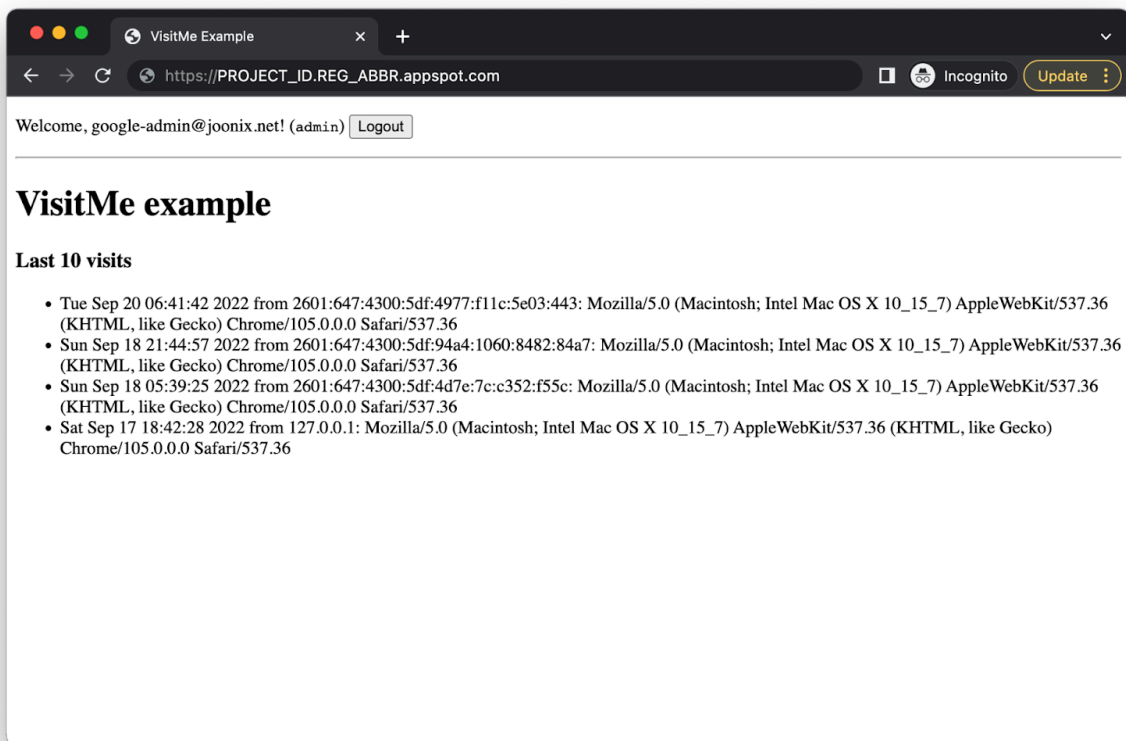
如果有多位使用者，系統會彈出帳戶選擇器對話方塊 (如下所示)。如果使用者尚未登入，系統會提示他們登入。如果已登入，彈出式視窗就會消失，應用程式也會進入登入狀態。



Module 21 的登入狀態與 Module 20 的使用者介面相同：



管理員使用者登入時也是如此：



與第 21 堂課不同，第 20 堂課一律會從應用程式 (伺服器端程式碼) 存取網頁範本內容的邏輯。Module 20 的缺點是，當使用者第一次點選應用程式時，系統會記錄一次造訪，使用者登入時則會記錄另一次造訪。

在第 21 個單元中，登入邏輯只會在網頁範本 (用戶端程式碼) 中執行。不需要進行伺服器端行程，即可判斷要顯示的內容。使用者登入後，系統只會呼叫伺服器一次，檢查使用者是否為管理員。也就是說，登入和登出不會計為額外造訪次數，因此使用者管理動作的最近造訪次數清單會維持不變。請注意，上述螢幕截圖顯示多個使用者登入時，都出現相同的四次造訪記錄。

本程式碼研究室開頭的「第 20 模組」螢幕截圖，說明瞭「重複造訪錯誤」。系統會為每次登入或登出動作顯示個別的造訪記錄。查看每個螢幕截圖的最新造訪時間戳記，瞭解時間順序。

清除所用資源

一般

如果暫時不需要使用，建議[停用 App Engine 應用程式](#)，以免產生費用。不過，如果您想進一步測試或實驗，App Engine 平台提供[免付費配額](#)，只要不超出該用量層級，就不會產生費用。這是指運算費用，但您可能也需要支付相關 App Engine 服務的費用，因此請參閱[其定價頁面](#)瞭解詳情。如果這項遷移作業涉及其他雲端服務，則這些服務會另外計費。無論是哪種情況，請視需要參閱下方的「本程式碼研究室專用」一節。

為求完整揭露，部署至 App Engine 等 Google Cloud 無伺服器運算平台時，會產生[少量建構和儲存空間費用](#)。Cloud Build 和 Cloud Storage 都有各自的免費配額。儲存該圖片會耗用部分配額。不過，你所在的區域可能沒有這類免付費層級，因此請留意儲存空間用量，盡量減少潛在費用。您應審查的特定 Cloud Storage「資料夾」包括：

- `console.cloud.google.com/storage/browser/LOC.artifacts.PROJECT_ID.appspot.com/containers/images`
- `console.cloud.google.com/storage/browser/staging.PROJECT_ID.appspot.com`
- 上述儲存空間連結取決於您的 PROJECT_ID 和 LOC，舉例來說，如果您的應用程式託管於美國，則為「us」。

另一方面，如果您不打算繼續使用這個應用程式或其他相關的遷移 Codelab，並想完全刪除所有內容，請[關閉專案](#)。

本程式碼研究室專用

下列服務是本程式碼研究室的專屬服務。詳情請參閱各項產品的說明文件：

- [App Engine Datastore](#) 服務是由 [Cloud Datastore](#) (Cloud Firestore Datastore 模式) 提供，這項服務也有免費方案；詳情請參閱[定價頁面](#)。
- 視您使用的服務而定，Cloud Identity Platform 具有一定程度的「免費」性質。詳情請參閱[定價頁面](#)。
- 根據[定價頁面](#)，Cloud Resource Manager API 大多可免費使用。

後續步驟

除了本教學課程外，您還可以參考其他遷移單元，瞭解如何從舊版套裝組合服務遷移：

- [單元 2](#)：從 App Engine `ndb` 遷移至 Cloud NDB
- [單元 7 至 9](#)：從 App Engine 工作佇列 (推送工作) 遷移至 Cloud Tasks
- [第 12 至 13 個單元](#)：從 App Engine Memcache 遷移至 Cloud Memorystore
- [單元 15 至 16](#)：從 App Engine Blobstore 遷移至 Cloud Storage
- [單元 18-19](#)：從 App Engine 工作佇列 (提取工作) 遷移至 Cloud Pub/Sub

App Engine 不再是 Google Cloud 中唯一的無伺服器平台。如果您有小型 App Engine 應用程式或功能有限的應用程式，並希望將其轉換為獨立的微服務，或是想將單體應用程式拆分成多個可重複使用的元件，這些都是考慮遷移至 [Cloud Functions](#) 的好理由。如果容器化已成為應用程式開發工作流程的一部分，特別是如果工作流程包含 CI/CD (持續整合/持續推送軟體更新或持續部署) 管道，建議遷移至 [Cloud Run](#)。下列單元會說明這些情況：

- 從 App Engine 遷移至 Cloud Functions：請參閱[第 11 堂課](#)
- 從 App Engine 遷移至 Cloud Run：請參閱[第 4 節](#)，瞭解如何使用 Docker 將應用程式容器化；或參閱[第 5 節](#)，瞭解如何在不使用容器、Docker 知識或 `Dockerfile`s 的情況下完成這項作業。

您可以選擇改用其他無伺服器平台，但建議先考量應用程式和用途的最佳選項，再進行任何變更。

無論您接下來要考慮哪個遷移模組，都可以在 [開放原始碼存放區](#) 存取所有 [Serverless Migration Station 內容](#) (程式碼研究室、影片、原始碼 [如有])。此外，該存放區的 `README` 也提供指引，說明要考慮哪些遷移作業，以及遷移模組的相關「順序」。

8. 其他資源

以下列出其他資源，供開發人員進一步瞭解這個或相關的遷移模組。您可以在下方提供這項內容的回饋、找到程式碼連結，以及各種實用說明文件。

程式碼研究室問題/意見回饋

如果發現本程式碼研究室有任何問題，請先搜尋問題，再提出回報。搜尋及建立新問題的連結：

- [搜尋問題](#)
- [回報新問題/提供意見](#)

遷移資源

下表提供第 20 課 (START) 和第 21 課 (FINISH) 的存放區資料夾連結。

Codelab	Python 2	Python 3
Module 20	code	(不適用)
第 21 個模組 (本程式碼研究室)	code	code

線上參考資料

以下是與本教學課程相關的資源：

Cloud Identity Platform 和 Cloud Marketplace

- [Identity Platform 產品頁面](#)
- [Firebase 驗證](#)
- [Identity Platform 和 Firebase 驗證產品比較頁面](#)
- [Identity Platform 定價資訊](#)
- [Identity Platform 配額 \(和無儀表板的使用量\)](#)
- [設定 Identity Platform 提供者](#)
- [Cloud Marketplace 產品頁面](#)
- [Marketplace 中的 Identity Platform 頁面](#)

Cloud Resource Manager、Cloud IAM、Firebase Admin SDK

- [Resource Manager 產品頁面](#)
- [Resource Manager 定價資訊](#)
- [Resource Manager 用戶端程式庫](#)
- [Cloud IAM 總覽 \(角色、允許政策等\)](#)
- [Firebase Admin SDK \(Python\)](#)

App Engine 使用者、App Engine NDB、Cloud NDB、Cloud Datastore

- [App Engine 使用者總覽](#)
- [App Engine NDB 說明文件](#)
- [App Engine NDB 存放區](#)
- [Cloud NDB 用戶端程式庫](#)
- [Cloud NDB 存放區](#)
- [Cloud Datastore 產品頁面](#)
- [Cloud Datastore 定價資訊](#)

其他遷移模組參考資料

- [遷移模組簡介](#)
- [所有「無伺服器遷移站」資源](#)
- [遷移至 Python 3 說明文件](#)
- [遷移單元 17「在第 2 代執行階段中使用套裝服務」程式碼研究室](#)
- [遷移模組 20「將 App Engine Users 服務新增至 Flask 應用程式」程式碼研究室](#)

App Engine 遷移

- [在 Python 2 應用程式中使用第三方程式庫](#)
- [第 2 代執行階段 \(Python 3\) 的 `app.yaml` 變更](#)
- [Cloud NDB 遷移指南](#)
- [Cloud NDB 遷移內容](#)

App Engine 平台

- [App Engine 說明文件](#)
- [Python 2 App Engine \(標準環境\) 執行階段](#)
- [在 Python 2 App Engine 上使用 App Engine 內建程式庫](#)
- [Python 3 App Engine \(標準環境\) 執行階段](#)
- [Python 2 和 3 App Engine \(標準環境\) 執行階段的差異](#)
- [Python 2 到 3 App Engine \(標準環境\) 遷移指南](#)
- [App Engine 定價和配額資訊](#)
- [推出第二代 App Engine 平台 \(2018 年\)](#)
- [比較第一代和第二代平台](#)
- [對舊版執行階段的長期支援](#)
- [說明文件遷移範例](#)
- [社群提供的遷移範例](#)

Cloud SDK

- [Google Cloud SDK](#)
- [Cloud SDK `gcloud` 指令列工具](#)
- [啟用 \(及停用\) Google API](#)
- [Cloud 控制台 API 管理員 \(啟用/停用 API\)](#)

- [使用 gcloud](#) 啟用 Google API
- [使用 gcloud](#) 列出 Google API

其他雲端資訊

- [在 Google Cloud 中使用 Python](#)
- [Python 用戶端程式庫說明文件](#)
- [Python 用戶端程式庫存放區](#)
- [「永久免費」級別](#)
- [Cloud SDK](#)
- [Cloud SDK gcloud 指令列工具](#)
- [所有 Google Cloud 說明文件](#)

影片

- [無伺服器遷移工作站](#)
- [無伺服器 Expeditions](#)
- [訂閱 Google Cloud Tech](#)
- [訂閱 Google Developers](#)

授權

這項內容採用的授權為 [Creative Commons 姓名標示 2.0 通用授權](#)。
